

TÍTULO DA ATIVIDADE: Entrega II de PI – Projeto11

PROFESSOR: Eduardo Savino Gomes

CURSO: Ciência da Computação

DISCIPLINA: Algoritmos e Lógica de Programação

TURMA: 1ºMACCOMP

DATA: 11/05/2026

ALUNO(A): Caio Moraes, Enzo Augusto Pedroza Gonçalves, Marcus Sousa, Nicolas Bologna, Lucas Elinou Feitosa

Player.cs

Essa parte do script Player.cs controla praticamente toda a lógica principal do jogo, incluindo movimentação do personagem, rotação da câmera em primeira pessoa, sistema de pontos, upgrades, autoclick, eventos de noite, troca de texturas, sistema de classes, interação com objetos através de Raycast e o sistema de rebirth.

Sistema de movimentação e câmera

Essa parte controla a movimentação do personagem utilizando CharacterController, permitindo deslocamento com colisão usando as teclas WASD. Também controla a rotação da câmera baseada no movimento do mouse, aplicando limites verticais para impedir rotações exageradas.

```
if (moveble == true)
{
    float moverVertical = Input.GetAxis("Vertical");
    float moverHorizontal = Input.GetAxis("Horizontal");

    Vector3 movimento = new Vector3(moverHorizontal, 0.0f, moverVertical);

    if (movimento.magnitude > 1)
    {
        movimento.Normalize();
    }

    if (characterController != null && characterController.enabled)
    {
        characterController.Move(transform.TransformDirection(movimento) *
        velocidade * Time.deltaTime);
    }
}
```

```
}  
}
```

Sistema de compra de upgrades

Essa parte do Player.cs controla a compra dos upgrades utilizando teclas do teclado. Cada melhoria possui um custo específico em pontos e altera atributos importantes do jogador como multiplicador de pontos, autoclick e limite máximo.

```
if (Input.GetKeyDown(KeyCode.H))  
{  
    if (pontos >= custoMulti)  
    {  
        pontos -= custoMulti;  
        multiplicadorPontos += 1;  
  
        custoMulti += 8;  
        precoMulti.text = "Preço: " + custoMulti;  
    }  
}
```

Sistema de Auto Click

Essa parte adiciona pontos automaticamente ao jogador conforme a quantidade de autoclicks comprados. O sistema utiliza um temporizador baseado em Time.deltaTime para controlar o intervalo entre cada ganho automático.

```
tempoAuto += Time.deltaTime;  
  
if (tempoAuto >= intervaloAuto)  
{  
    tempoAuto = 0f;  
  
    pontos += clicksAuto * clicksautomaticosclasse;  
  
    pontos = Mathf.Clamp(pontos, 0, pontosMaximos);  
}
```

Sistema de interação por Raycast

Essa parte utiliza Raycast para detectar objetos que o jogador está olhando e clicando. Dependendo do objeto atingido, diferentes ações são executadas, como clicar no computador, ligar luzes, ativar bônus ou iniciar o rebirth.

```

Ray ray = cameraTransform
    .GetComponent<Camera>()
    .ViewportPointToRay(new Vector3(0.5f, 0.5f, 0));

RaycastHit hit;

if (Physics.Raycast(ray, out hit, maxDistance, hitLayers))
{
    Debug.Log("Acertou: " + hit.collider.gameObject.name);
}

```

Sistema de evento de noite

Essa parte controla o evento aleatório de noite do jogo. Após determinado tempo, existe uma chance do evento ser ativado, alterando iluminação, multiplicadores e liberando bônus temporários.

```

if (tempoNoite >= 300f)
{
    tempoNoite = 0f;

    bool ativarEvento = primeiroEventoNoite || Random.Range(0f, 100f) <=
chance;

    if (ativarEvento)
    {
        eventoNoiteAtivo = true;
        tempoEventoNoite = 0f;
        luzSol.intensity = 0f;
    }
}

```

Sistema de texturas

Essa parte permite ao jogador trocar o pacote de texturas do quarto, alterando materiais de paredes, chão, cama, porta e janela. O jogo possui pacotes padrão, realista, monocromático e hyperpop.

```

public void TexturasMono()
{
    portaRenderer.material.mainTexture = portamono;

    parede1Renderer.material = materialparedemono;
    parede2Renderer.material = materialparedemono;

    chaoRenderer.material = materialchaomono;
}

```

```
mono = 1;
realista = 0;
hyperpop = 0;
}
```

Sistema de classes

Essa parte permite ao jogador selecionar classes diferentes, alterando atributos específicos do gameplay. Cada classe modifica multiplicadores, autoclicks automáticos ou limite máximo de pontos.

```
if (Input.GetKeyDown(KeyCode.Alpha5))
{
    ClasseTexto.text = "Classe: Python";

    multiplicadorClasse = 1;
    clicksautomaticosclasse = 5;
    limiteclasse = 1;
}
```

Sistema de animação do clique

Essa parte executa uma animação no texto de pontos sempre que o jogador clica no computador. Além da animação, também atualiza os pontos atuais.

```
Animator animClick = textoPontos2.GetComponent<Animator>();

if (animClick != null)
{
    animClick.enabled = true;

    animClick.Rebind();
    animClick.Update(0f);

    animClick.Play("animacaoPontos");
}
```

Sistema de Rebirth

Essa parte controla o sistema de rebirth do jogo. Quando o jogador alcança a quantidade necessária de pontos, é possível iniciar uma cutscene e resetar todo o progresso mantendo a progressão do gameplay.

```
public IEnumerator Rebyrth()
{
    HUDManager.hudPrincipal.gameObject.SetActive(false);

    canvascutscene.SetActive(true);

    videoCutscene.Stop();
    videoCutscene.frame = 0;

    videoCutscene.Play();

    yield return null;
}
```

Sistema de reset completo

Essa parte redefine todos os dados principais do jogo após o rebirth, incluindo pontos, upgrades, classes, texturas, eventos, iluminação e HUD.

```
public void ResetarJogo()
{
    pontos = 0;
    multiplicadorPontos = 1;

    clicksAuto = 0;

    pontosMaximos = 500;

    custoMulti = 12;
    custoAuto = 6;

    chance = 7f;

    moedapaga = 0;

    TexturasPadrao();
}
```

HUDTutorial.cs

Essa parte do HUDTutorial.cs é responsável por controlar as mensagens tutoriais do jogo, apresentando explicações sobre mecânicas importantes como upgrades, autoclick, evento noturno, cafeteira e sistema de rebirth.

HUDManager.cs

Essa parte do HUDManager.cs controla a ativação e desativação das interfaces do jogo, gerenciando menus principais, upgrades, inventário, texturas, classes e telas secundárias.

ClickSpawner.cs

Essa parte do ClickSpawner.cs controla a criação de cliques visuais e o sistema de multiplicador aplicado aos pontos recebidos pelo jogador.

Sistema de iluminação

Essa parte controla as luzes do quarto, do computador e do ambiente externo, alterando intensidade e materiais da janela dependendo do ciclo de dia e noite.

```
if (luzSol.intensity == 0f && luzQuarto.intensity == 0f)
{
    luzComputador.intensity = 500f;
}
else
{
    luzComputador.intensity = 0f;
}
```

Sistema de vídeos e créditos

Essa parte controla a reprodução da cutscene final utilizando VideoPlayer, além de executar os créditos do jogo ao término do vídeo.

```
void FimDoVideo(VideoPlayer vp)
{
    canvascutscene.SetActive(false);
    HUDManager.hudPrincipal.gameObject.SetActive(true);

    ResetarJogo();

    StartCoroutine(hudcreditos.Creditos());
}
```

Considerações Finais

O Projeto11 utiliza uma arquitetura baseada em scripts responsáveis por sistemas específicos do jogo, facilitando manutenção, expansão e organização do código. O Player.cs concentra grande parte da lógica principal do gameplay, enquanto scripts auxiliares controlam HUD, tutoriais, eventos, iluminação, texturas e progressão do jogador.

O projeto utiliza diversos conceitos fundamentais de programação em Unity e C#, como:

- Raycast
- Coroutines
- Manipulação de UI
- Sistema de Input
- Controle de iluminação
- VideoPlayer
- Manipulação de materiais
- Sistema de progressão
- Programação orientada a objetos
- Eventos temporizados com Time.deltaTime
- Organização modular de scripts